Smart Inventory Demand Forecasting Decision Support System

1st Kavyashri NP
Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
kavyashri.np@cmrit.ac.in

2nd Pavan Raj B M
Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
pabm23cs@cmrit.ac.in

3rd Pavan BV
Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
pabv23cs@cmrit.ac.in

4th Ranjith Bhandari
Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
rabh23cs@cmrit.ac.in

***Abstract*—Inventory planning requires a balance between product availability and the financial exposure associated with excess stock. Forecasting demand alone is insufficient when operational users must translate predictions into replenishment decisions, assess uncertainty, compare scenarios, and retain accountability for procurement actions. This paper presents a modular web-based Smart Inventory Demand Forecasting Decision Support System that connects supervised demand forecasting with inventory-risk calculations, explainability, scenario analysis, and a human-in-the-loop approval workflow. The implemented prototype uses a 48-row chronological dataset covering four products over twelve months and evaluates Linear Regression, Ridge Regression, Decision Tree, and Random Forest models using a 75:25 time-ordered split. In the reproducible baseline experiment, Ridge Regression obtained the lowest MAE of 13.39 and MAPE of 1.70%, while Linear Regression produced the highest R^2 of 0.9935. Ridge was therefore selected under the predefined MAE-first ranking criterion. The system further estimates forecast uncertainty, computes stockout and overstock indicators, derives safety stock and reorder points, supports what-if changes to demand-related inputs, and records managerial approval decisions. Because only 12 observations form the holdout set, the results are treated as preliminary evidence of prototype feasibility rather than evidence of general commercial forecasting performance. The contribution is consequently positioned as a practical forecast-to-decision integration rather than a new forecasting algorithm.**

***Index Terms*—Demand forecasting, inventory management, decision support system, machine learning, Ridge regression, supply chain analytics, explainable AI, inventory optimization.**

I. Introduction

Inventory decisions are made under uncertainty: demand can change while inventory is already committed, supplier lead times can constrain replenishment, and price or sales conditions can alter the relationship between observed transactions and future demand. An under-estimated requirement can result in stockouts, delayed fulfilment, and lost revenue, whereas an over-estimated requirement can increase holding costs and tie up working capital. Recent research has therefore moved beyond isolated demand prediction toward approaches that connect predictive information with inventory decisions and supply-chain performance [1], [2], [4], [5].

Machine-learning methods provide a flexible way of representing relationships among historical sales, inventory position, price, time, and supplier information. Recent demand-forecasting studies have investigated feature engineering, ensemble learning, deep learning, and hybrid architectures for heterogeneous product behaviour [3], [9], [10], [12]. At the inventory-control level, the literature increasingly considers learning methods as either a separate forecasting stage followed by optimization or as part of an integrated decision policy [1], [5], [8]. These developments indicate that forecasting quality is important, but operational usefulness also depends on how predictions are interpreted and converted into actions.

Explainability is particularly relevant when a forecast contributes to a purchasing recommendation. Supply-chain decision makers may need to understand which variables influenced a prediction and how a change in demand or lead time would affect risk. Research on explainable artificial intelligence (XAI) in supply-chain decision support emphasizes the role of transparent predictive evidence in managerial decision making [6], [7]. At the same time, recent integrated approaches based on deep learning and reinforcement learning demonstrate the potential of more complex forecasting-to-control pipelines [9], [14].

The present work takes a deliberately lightweight systems approach. Instead of introducing another forecasting architecture, it integrates four supervised regression models with a web application, a deterministic inventory decision layer, empirical uncertainty estimates, feature-sensitivity analysis, scenario simulation, and human approval. The resulting workflow is intended to support managerial judgement rather than replace it. This design also makes the assumptions of the inventory calculations visible to users.

The main contributions are: (1) a chronological multi-model

demand-forecasting pipeline; (2) integration of forecast outputs with safety-stock, reorder-point, and order-quantity calculations; (3) empirical uncertainty and baseline-perturbation feature contribution estimates; (4) scenario-based sensitivity analysis; (5) human-in-the-loop recommendation approval with audit logging; and (6) a decoupled web architecture linking Node.js application services with Python machine-learning inference. The work is evaluated as a prototype and does not claim state-of-the-art forecasting performance or production-scale generalization.

The remainder of the paper reviews relevant literature, defines the problem and objectives, describes the architecture and methodology, reports the baseline experiment, discusses decision-support functions and limitations, and concludes with future research directions.

II. Literature Review

A. Machine Learning for Demand Forecasting

Demand forecasting research has expanded from classical statistical models toward machine learning and deep learning. Mejía and Aguilar proposed a demand-forecasting system that first characterizes product time series, clusters products according to demand behaviour, and then evaluates forecasting methods and ensembles for the resulting groups [3]. Their work illustrates the importance of product heterogeneity and feature engineering. Zhang **et al.** investigated hybrid attention-based deep-learning models for time-series product demand, reflecting the continued development of sequence-oriented forecasting methods [9]. Riachy **et al.** addressed demand forecasting under large data gaps, showing that data availability and feature engineering remain important methodological concerns [10].

These studies motivate comparative model evaluation rather than assuming that one algorithm is universally suitable. The present prototype therefore evaluates four conventional supervised regressors using the same chronological holdout. The objective is not to outperform deep-learning systems, but to determine which lightweight model is most appropriate for the available prototype data and then connect that model to inventory decision logic.

B. Inventory-Aware Forecasting

The separation between forecasting and inventory control has increasingly been challenged. van der Haar **et al.** investigated supervised learning for integrated forecasting and inventory control and demonstrated that feature information can be incorporated directly into learning-based inventory decisions [1]. Sukolkit **et al.** studied demand forecasting for steel inventory management and compared several forecasting architectures before integrating the forecasting outcome with inventory strategies [4]. Abdi **et al.** considered demand forecasting within multi-objective, multi-product, and multi-period supply-chain optimization [8]. Sarıcıoğlu **et al.** examined one-step and multi-step forecasting in relation to the bullwhip effect and supply-chain performance [11].

Bergsma **et al.** systematically reviewed machine-learning approaches in inventory-control optimization and distinguished approaches that use ML for separate estimation, integrated optimization, and dynamic policy learning [5]. This classification places the current prototype closest to the separate-estimation/inventory-decision integration category: demand is forecast first, and the resulting estimate is passed to deterministic inventory logic. This is intentionally simpler than end-to-end inventory-policy learning.

C. Explainable AI and Decision Support

Explainability becomes important when predictions influence business decisions. Jauhar **et al.** examined explainable AI for resilient perishable-product supply chains using customer characteristics and transparent machine-learning techniques [6]. Olana **et al.** reviewed the role of XAI capabilities in supply-chain decision-support systems and emphasized the importance of explanations in decision contexts [7]. The present system does not implement formal Shapley-value attribution; instead, it uses a baseline-perturbation mechanism to estimate feature sensitivity. This distinction is retained explicitly because approximate feature contributions should not be interpreted as formal SHAP guarantees.

D. Research Gap

The literature shows a practical gap between demand prediction and auditable inventory action. Forecasting studies often emphasize predictive accuracy, while integrated optimization and reinforcement-learning approaches can be unnecessarily complex for small prototype datasets. Explainability and uncertainty are also frequently presented as analytical outputs rather than as evidence within a human review process. The proposed system addresses this integration gap by connecting lightweight forecasting, inventory-risk logic, scenario analysis, approval, and auditability without claiming novelty in the individual algorithms.

III. Problem Statement and Objectives

Retail and warehouse operators must replenish inventory while demand, prices, stock levels, and supplier lead times change. A forecast alone does not provide sufficient evidence for procurement, while a fully autonomous policy can ignore contextual information unavailable in historical data. The problem addressed here is therefore to transform historical observations into demand estimates and interpretable replenishment evidence while preserving human authority over the final action.

A. Objectives

The system aims to validate inventory-demand data; compare supervised regressors under chronological evaluation; estimate demand from sales, inventory, price, month, and lead time; expose forecast metrics and uncertainty; compute safety stock, reorder point, coverage and order quantity; support what-if analysis; provide feature-sensitivity explanations; and record approval, rejection, or snooze decisions.

B. Contributions

The contribution is system-level integration rather than a new learning algorithm. A conventional forecasting pipeline is connected to deterministic inventory reasoning, scenario analysis, and an auditable web workflow, allowing the forecasting component to be replaced without redesigning the decision layer.

IV. System Architecture and Software Engineering

The application uses a decoupled browser frontend, Node.js/Express backend, JSON persistence layer, and Python machine-learning process. The backend invokes Python through child-process execution and exchanges structured JSON payloads. JWT authentication and role checks protect forecasting and procurement operations. The local prototype stores users, datasets, scenarios, actions, suppliers, purchase requests, purchase orders, receipts, transactions, and audit records in JSON files; this avoids a native database dependency but is not evidence of production-scale persistence.

The frontend provides demand and inventory intelligence, scenario analysis, model comparison, data-quality, action-review, audit, and reporting views, with Chart.js for visualization. REST endpoints expose authentication, forecasting, intelligence, scenario, audit, and procurement functions. The Python module uses pandas, NumPy, and scikit-learn, with sales, inventory, price, month, and lead time as the baseline feature vector and demand as the target. Model caching avoids repeated training when data and selected features are unchanged.

Forecast output enters a deterministic decision layer that derives predicted demand, stockout risk, safety stock, reorder point, inventory coverage, revenue-at-risk information, and recommendation categories. Scenario requests alter demand-related inputs, price, inventory, or lead time and re-run inference without overwriting the baseline dataset. Recommendations remain subject to APPROVE, REJECT, or SNOOZE review; contextual reasons and decision metadata are retained, while the procurement module supports suppliers, requests, orders, receipts, and inventory transactions.

V. Mathematical Formulation and Methodology

A. Dataset and Feature Engineering

The supplied baseline dataset contains 48 observations representing four products across twelve chronological months. The CSV contains product, sales, inventory, price, month, demand, lead-time, and category fields. For the reported multi-model comparison, the numerical forecasting features are sales, inventory, price, month, and lead time. Product and category fields are retained in the dataset but are not used as predictors in the baseline comparison. The target is unit demand.

The chronological order is preserved. With a 75:25 split, 36 observations are used for training and 12 for testing. This arrangement avoids random shuffling and prevents future observations from being used to train the reported holdout evaluation.

B. Data Quality Assessment

The quality module checks missing values, duplicate rows, invalid numeric values, negative values, required columns, and numerical outliers. Outliers are identified using the interquartile range (IQR):

$$\mathrm{IQR} = Q_3 - Q_1.$$

A value below $Q_1 - 1.5\mathrm{IQR}$ or above $Q_3 + 1.5\mathrm{IQR}$ is flagged. The quality score is an engineering diagnostic and is not treated as a statistical measure of dataset validity.

C. Preprocessing and Temporal Evaluation

The baseline comparison sorts the data by month and applies the chronological split before model fitting. Linear Regression and Ridge use standardized numerical features. The preprocessing parameters for these models are fitted from the training portion and then applied to the test portion. The chronological split function uses the first 75% of ordered observations for training and the remaining 25% for testing.

D. Regression Models

For Linear Regression, the parameter vector minimizes the residual sum of squares:

$$\min_w |Xw - y|_2^2.$$

Ridge Regression adds an L_2 penalty:

$$\min_w |Xw - y|_2^2 + \alpha |w|_2^2,$$

where the implementation uses $\alpha = 1.0$. Decision Tree Regression uses a maximum depth of five. Random Forest uses 100 trees for the reported comparison experiment, with a fixed random seed of 42.

The production-style forecast path also supports a 200-tree Random Forest for individual inference. This configuration difference is important: the 100-tree model is the one used for the reported four-model comparison, whereas the 200-tree configuration is an inference configuration. The two should not be treated as the same experiment.

E. Evaluation Metrics

Four standard regression metrics and forecast bias are reported. Mean Absolute Error is

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|.$$

Root Mean Squared Error is

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}.$$

For nonzero targets, MAPE is

$$\mathrm{MAPE} = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|.$$

The coefficient of determination is

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

Forecast bias is defined by the implementation as $\mathrm{mean}(\hat{y} - y)$, so negative values indicate under-forecasting on average.

F. Forecast Uncertainty

For Random Forest, the implementation calculates dispersion across individual tree predictions and combines it with training residual variability. For a nominal 90% level, the interval is represented as

$$U_{90\%} = \hat{y} \pm 1.645\sigma,$$

where σ represents the combined empirical uncertainty estimate. For non-tree models, residual standard deviation from chronological training residuals is used. These values are presented as empirical uncertainty estimates, not statistically calibrated prediction intervals, because coverage has not been validated on independent repeated samples.

G. Feature Contribution Analysis

The explanation mechanism is a baseline perturbation procedure. A baseline feature vector is formed from the mean feature values. The prediction at the baseline is compared with predictions generated after replacing one feature with the active observation while keeping other baseline values fixed. The difference is reported as a feature contribution. This method provides local sensitivity information but is not equivalent to exact Shapley-value decomposition.

H. Inventory Decision Logic

The forecast is converted from a monthly quantity to an approximate daily rate:

$$\bar{d} = \frac{\hat{D}}{30}.$$

Under the simplifying assumption that supplier lead-time variability is negligible, safety stock is represented as

$$SS = Z\sigma_d\sqrt{\bar{L}},$$

where Z is the service-level factor, σ_d is demand variability, and $\bar{L}$ is mean lead time. For a nominal 95% service level, $Z = 1.65$. The reorder point is

$$ROP = \bar{d}\bar{L} + SS.$$

The prototype order-quantity rule is

$$Q = \max(0, \hat{D} + SS - I),$$

where I is current inventory. This is a deterministic prototype policy, not a cost-optimal multi-echelon inventory-control formulation.

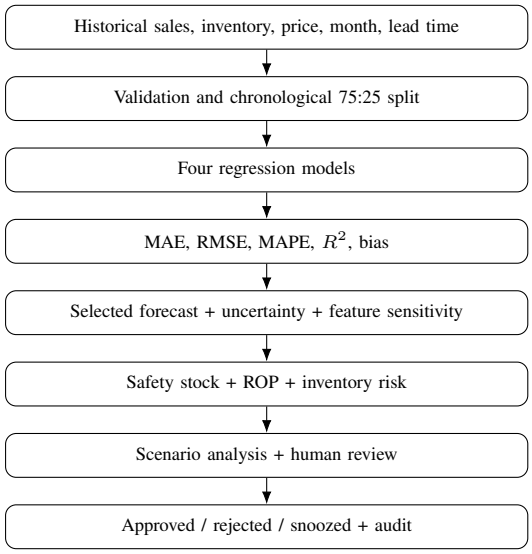


Fig. 1. Forecast-to-decision workflow.

VI. Experimental Setup and Results

A. Hardware and Software Environment

The project is implemented as a full-stack web application. The backend uses Node.js 18+ and Express, while the machine-learning module uses Python with pandas, NumPy, and scikit-learn. The frontend is delivered as a browser application using HTML, CSS, JavaScript, and Chart.js. JSON files provide local persistence during development. The reported experiment is therefore a prototype evaluation rather than a distributed production benchmark.

B. Experimental Protocol

The baseline CSV contains 48 rows. The Python comparison routine sorts the records by month, separates the first 36 observations from the final 12 observations, trains all four regressors on the training portion, and computes metrics only on the holdout portion. The experiment is deterministic with fixed model random seeds where applicable. No random train-test shuffling is used.

C. Model Configuration

Linear Regression and Ridge Regression use standardized numerical inputs; Ridge uses $\alpha = 1.0$. Decision Tree Regression uses maximum depth five. The reported Random Forest comparison uses 100 estimators and random seed 42; the separate inference path may use 200 trees.

D. Model Comparison

The reproducible baseline comparison is shown in Table I. Ridge Regression obtains the lowest MAE and MAPE, while Linear Regression obtains the lowest RMSE and slightly higher R^2. The selection procedure ranks models primarily by MAE, followed by RMSE, MAPE, R^2, and absolute bias. Under this predefined ordering, Ridge is selected.

TABLE I
REGRESSION PERFORMANCE ON THE 12-OBSERVATION HOLDOUT SET

Model	MAE	RMSE	MAPE(%)	R^2	Bias
Random Forest	48.93	66.34	5.67	0.8955	-16.03
Linear Regression	13.87	16.58	2.00	0.9935	-10.57
Ridge Regression	**13.39**	17.70	**1.70**	0.9926	-9.95
Decision Tree	37.08	47.76	4.52	0.9458	-5.42

E. Error Analysis and Model Selection

Ridge Regression improves on Linear Regression in MAE and MAPE, but not in RMSE or R^2. The difference is therefore criterion-dependent rather than evidence that Ridge dominates every measure. The model selection is intentionally based on MAE because MAE expresses the average forecasting error in the original demand units and is easier to relate to inventory planning.

The tree-based models perform substantially worse on the holdout set. With only 36 training observations, non-parametric models have limited evidence from which to learn stable partitions of the feature space. Ridge introduces regularization and therefore imposes stronger structural constraints, which can be advantageous when data are scarce and predictors are correlated. This interpretation is specific to the current experiment and should not be generalized to larger commercial datasets.

F. Interpretation Caveat

The reported R^2 values above 0.99 for the two linear models should be interpreted cautiously. They arise from a very small, structured dataset and a single 12-observation test window. The experiment demonstrates that the implemented data flow can train models, compare their errors, select a model, and pass predictions to the decision layer. It does not establish that the same error rates would be obtained on larger, noisier, seasonal, or highly volatile inventory datasets.

VII. DECISION SUPPORT FEATURES

A. Demand Intelligence and Inventory Risk

The dashboard combines the forecast with inventory context. The decision layer derives safety stock, reorder point, inventory coverage, stockout probability, and revenue-at-risk information. This allows a user to interpret a forecast in relation to current stock rather than treating the forecast as an isolated number. Recommendation states can include `REORDER_NOW`, `REORDER_SOON`, `HOLD`, `REDUCE_ORDER`, and `MONITOR`, subject to the implemented risk rules.

B. Scenario Simulation

The scenario engine accepts controlled changes to demand-related inputs, including sales or demand assumptions, price, inventory, and lead time. The modified feature vector is passed through the forecasting process and the resulting demand and risk indicators are compared with the baseline. Importantly, scenario changes are treated as hypothetical inputs and do not overwrite the underlying baseline dataset.

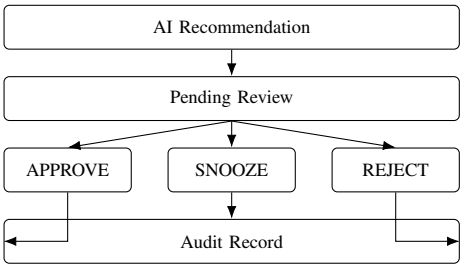


Fig. 2. Human-in-the-loop action and audit workflow.

C. Human-in-the-Loop Approval

A recommendation is placed into an action workflow rather than immediately creating an autonomous procurement action. The user can approve, reject, or snooze the recommendation. A reason can be captured for rejection or delay. This mechanism is useful because managers may possess information about promotions, supplier relationships, upcoming events, or business constraints that is not represented in the historical feature set.

D. Auditability and Procurement

The backend records actions in persistent JSON structures and supports procurement entities including suppliers, purchase requests, purchase orders, goods receipts, and inventory transactions. Purchase orders are protected by approval and role checks. Status transitions are explicitly controlled, allowing the system to preserve a trace of procurement progress. These capabilities extend the forecasting prototype toward a more complete operational workflow while remaining dependent on the limitations of local JSON persistence.

VIII. CHALLENGES AND LIMITATIONS

The principal limitation is the small evaluation dataset: 48 observations with only 12 in the holdout window are insufficient to establish reliable multi-year seasonality, promotion effects, structural breaks, or broad market volatility. A single chronological holdout is also sensitive to the selected test period. The current formulation does not explicitly correct for censored demand during stockouts and uses a limited set of explanatory variables without holidays, promotions, competitor prices, macroeconomic indicators, or external demand signals.

JSON persistence is suitable for local demonstration but lacks the concurrency, transactions, indexing, backup, and access controls expected from a production database. Uncertainty estimates are empirical rather than calibrated, and the feature-attribution mechanism is a baseline-perturbation approximation rather than exact SHAP. Finally, the order-quantity rule is deterministic and does not optimize holding cost, stockout penalty, supplier capacity, multi-echelon interactions, or service-level trade-offs. The system should therefore be understood as decision support rather than a complete inventory-control optimizer.

IX. FUTURE RESEARCH DIRECTIONS

Future work should use larger multi-year SKU datasets and rolling-origin or walk-forward validation to provide several

independent test periods. The software can migrate from JSON to PostgreSQL or another transactional database and incorporate promotions, holidays, stockout indicators, supplier reliability, and external market signals. With sufficient data, LSTM/GRU, attention-based, time-series mixer, and reinforcement-learning approaches can be compared against the current regression baseline under a common temporal protocol. Prediction intervals should be calibrated empirically and formal explainability methods such as SHAP evaluated against the current perturbation approximation.

Inventory control can also progress from the deterministic replenishment rule toward cost-aware optimization incorporating holding cost, stockout cost, service-level constraints, supplier capacity, and multi-echelon interactions.

X. Conclusion

This paper presented a Smart Inventory Demand Forecasting Decision Support System integrating supervised demand forecasting, inventory-risk calculations, scenario simulation, feature-sensitivity information, and a human-in-the-loop workflow. The implementation connects a browser frontend to Node.js/Express and Python scikit-learn inference, with lightweight JSON persistence for prototype data and audit records.

On the 48-row baseline dataset, four regressors were compared using a chronological 75:25 split. Ridge Regression achieved the lowest MAE of 13.39 and MAPE of 1.70%, while Linear Regression obtained a slightly higher R^2 of 0.9935 and lower RMSE than Ridge. The results are preliminary because only twelve observations form the holdout set. The principal contribution is therefore the integration of forecasting into an auditable decision-support workflow; larger datasets, repeated temporal validation, calibrated uncertainty, richer demand variables, and cost-aware inventory optimization are required before production-scale or generalization claims can be justified.

References

[1] J. F. van der Haar, A. P. Wellens, R. N. Boute, and R. J. I. Basten, "Supervised learning for integrated forecasting and inventory control," **European Journal of Operational Research**, vol. 319, no. 2, pp. 573–586, 2024.

[2] L. Bo and J. Xu, "Enhancing supply chain efficiency resilience using predictive analytics and computational intelligence techniques," **IEEE Access**, vol. 12, pp. 183451–183465, 2024.

[3] S. Mejía and J. Aguilar, "A demand forecasting system of product categories defined by their time series using a hybrid approach of ensemble learning with feature engineering," **Computing**, vol. 106, pp. 3945–3965, 2024.

[4] N. Sukolkit, S. Arunyanart, and A. Apichottanakul, "An open innovative inventory management based demand forecasting approach for the steel industry," **Journal of Open Innovation: Technology, Market, and Complexity**, vol. 10, no. 4, Art. no. 100407, 2024.

[5] R. Bergsma, C. de Ruijt, and S. Bhulai, "A systematic review of machine learning approaches in inventory control optimization," **Operations Research Perspectives**, vol. 15, Art. no. 100367, 2025.

[6] S. K. Jauhar, S. Harinath, V. Krishnaswamy, and S. K. Paul, "Explainable artificial intelligence to improve the resilience of perishable product supply chains by leveraging customer characteristics," **Annals of Operations Research**, vol. 354, pp. 103–142, 2025.

[7] F. Olana, K. Spanaki, W. Ahmed, and G. Zhao, "Enabling explainable artificial intelligence capabilities in supply chain decision support making," **Production Planning & Control**, pp. 808–819, 2024.

[8] F. Abdi **et al.**, "A machine learning-based approach for multi-objective, multi-product, and multi-period supply chain optimization via demand forecasting," **IEEE Access**, vol. 13, pp. 147343–147368, 2025.

[9] X. Zhang **et al.**, "Enhancing time series product demand forecasting with hybrid attention-based deep learning models," **IEEE Access**, vol. 12, pp. 190079–190091, 2024.

[10] C. Riachy **et al.**, "Enhancing deep learning for demand forecasting to address large data gaps," **Expert Systems with Applications**, vol. 268, Art. no. 126200, 2025.

[11] A. Sarıcıoğlu, M. E. Genevois, and M. Cedolin, "Analyzing one-step and multi-step forecasting to mitigate the bullwhip effect and improve supply chain performance," **IEEE Access**, vol. 12, pp. 180161–180174, 2024.

[12] T. N. A. Nguyen **et al.**, "Optimizing supply chain operations using advanced time-series mixer models for demand forecasting and inventory under uncertain demand," **Expert Systems with Applications**, vol. 296, Art. no. 128955, 2026.

[13] A. K. Mukherjee **et al.**, "Machine learning-driven demand forecasting for multi-objective sustainable inventory control under uncertainty," **International Journal of Management Science and Engineering Management**, 2026.

[14] Y. Yang **et al.**, "Multi-agent deep reinforcement learning for integrated demand forecasting and inventory optimization in sensor-enabled retail supply chains," **Sensors**, vol. 25, no. 8, Art. no. 2428, 2025.

[15] R. Khan, "AI-enabled predictive forecasting for inventory optimization in pharmaceutical supply chains," **World Journal of Advanced Engineering Technology and Sciences**, vol. 17, no. 2, 2025.